

INFORME DE REVISION TECNICA

Webapp Venta de Autopartes

Verificacion de consigna, bugs y errores generales

Material revisado	Proyecto.Cursada2024.pdf + codigo fuente del proyecto
Tecnologias	Laravel 10, API REST, React, Blade, PostgreSQL / configuracion de despliegue
Fecha del informe	25 de agosto de 2026

Diagnostico general

CONSIGNA Cumplimiento parcial	HALLAZGOS 6 criticos / 6 altos / 7 medios	ENTREGA FINAL Requiere correcciones
VEREDICTO La base del proyecto esta desarrollada y cubre buena parte del flujo de e-commerce, pero no puede considerarse una entrega final sin correcciones. Los bloqueantes principales son la autorizacion por roles, el backend administrativo Blade, la transaccion de compra y la configuracion de produccion.		

Fuente de la consigna: Proyecto.Cursada2024.pdf, paginas 1-2.

1. Resumen ejecutivo

La aplicacion implementa una tienda de autopartes con Laravel, API REST y React. La estructura general coincide con el enfoque full-stack solicitado: administracion de datos en Laravel, una API intermedia y un frontend orientado al usuario final. Sin embargo, la revision encontro problemas que afectan tanto el cumplimiento de la consigna como la seguridad y la robustez del sistema.

PUNTO CENTRAL

No es posible afirmar ausencia de bugs. Hay fallas reproducibles o deducibles del codigo que deben corregirse antes de presentar el proyecto como terminado. Ademas, la cobertura de tests existente es demasiado limitada para sostener una afirmacion de "sin errores".

La consigna establece, entre otros puntos, que el proyecto sea un e-commerce, que el backend se implemente en Laravel con interfaz Blade para administradores, que el frontend sea React o Vue para el publico general y que el acceso a los datos se realice mediante una API REST. En el ejemplo de venta de productos, tambien se espera ABM administrativo de productos/promociones, consulta de productos y registro de pedidos de compra.

2. Cumplimiento de la consigna

Requisito	Estado	Observacion
Aplicacion de comercio electronico	CUMPLE	Es una tienda de autopartes y dispone de carrito / flujo de compra.
Backend Laravel	CUMPLE	El backend esta implementado sobre Laravel 10.
Backend administrativo con Blade	NO CUMPLE	Existen vistas Blade, pero el flujo administrativo web esta roto o no conectado correctamente.
ABM de productos	PARCIAL	La logica existe, pero el flujo utilizable esta concentrado principalmente en API / React.
API REST intermedia	PARCIAL	Existe una API REST; sin embargo, las imagenes se consumen directamente desde /storage, generando acoplamiento con Laravel.
Frontend React o Vue	CUMPLE	El frontend esta desarrollado en React.
Frontend para usuario final	PARCIAL	Productos, carrito y pedidos estan presentes, pero el ABM de empleados tambien se expone dentro del frontend React.
Carrito y registro de compra	PARCIAL	La funcionalidad existe, aunque la transaccion de compra presenta un bug serio ante falta de stock.
Interfaz atractiva, responsive y facil de usar	PARCIAL	Se usa Bootstrap y existen media queries; no se pudo completar una validacion visual integral del build.
Backend administrativo no publico	NO CUMPLE	El registro publico permite seleccionar Empleado y la API acepta ese rol.
Deploy de produccion	NO CUMPLE	La configuracion analizada conserva URLs y CORS de localhost.
GitHub, PR y checkpoints	NO VERIFICABLE	No puede comprobarse de forma concluyente a partir del ZIP analizado.

Referencia de consigna: pagina 1 para arquitectura full-stack, e-commerce, Laravel + Blade, API y React/Vue; pagina 2 para frontend publico, uso de API REST, checkpoints y el ejemplo de venta de productos.

3. Bugs y errores encontrados

Severidad	Hallazgo	Impacto / descripcion	Ubicacion
CRITICO	Registro publico como Empleado	El formulario publico permite elegir Cliente o Empleado y la API acepta ambos. Esto permite intentar obtener un rol administrativo desde un punto de entrada publico.	frontend/src/pages/Register.jsx:321-342; RegisterController.php:55
CRITICO	API sin autorizacion por rol para ABM	Un usuario con token valido puede invocar POST/PUT/DELETE sobre autopartes sin una comprobacion de rol en las rutas protegidas. Ocultar botones en React no reemplaza la autorizacion del servidor.	backend/routes/api.php:14-20
CRITICO	Roles Blade con mayusculas/minusculas incompatibles	Los roles se guardan como Cliente/Empleado, mientras las rutas usan role:cliente y role:empleado. CheckRole compara literalmente, por lo que el acceso puede fallar aunque el usuario tenga el rol esperado.	web.php:27,50; CheckRole.php:20
CRITICO	Rutas de Cliente incluyen editar y eliminar productos	Al corregir el problema de casing, el grupo Cliente conserva rutas de modificacion y borrado de autopartes. Un cliente no deberia tener permisos administrativos.	web.php:44-46
CRITICO	Rutas Blade apuntan a metodos inexistentes	Se detectaron siete entradas de rutas que refieren a acciones inexistentes. Esto impide considerar completo y operativo el backend administrativo Blade.	web.php; varios controladores
CRITICO	Checkout puede confirmar una compra parcialmente fallida	El Pedido se crea antes de validar todo el stock. Si falta stock se retorna un response 400 dentro de DB::transaction sin lanzar excepcion, por lo que la transaccion puede llegar a confirmarse igualmente.	CompraController.php:23-91
ALTO	Configuracion de produccion hardcodeada a localhost	La URL de API, CORS y rutas de imagen contienen 127.0.0.1. En un deploy real el frontend puede no comunicarse correctamente con el backend.	frontend/src/api.js:4; config/cors.php:23-25; Autopartes.jsx:171; DetalleAutoparte.jsx:136
ALTO	Logout incompleto	Se elimina el objeto user pero el auth_token permanece. El interceptor sigue enviando el token y la sesion API continua autenticada. Tampoco se observo una revocacion de token en backend.	Autopartes.jsx:122-123 y paginas relacionadas
ALTO	Riesgo de secretos y archivos de entorno	El ZIP contiene .env y .env_production con configuracion sensible. Ademas, .gitignore ignora .env.production pero no .env_production.	backend/.gitignore:8-10; archivos .env

Severidad	Hallazgo	Impacto / descripcion	Ubicacion
ALTO	APP_DEBUG activo en produccion	La configuracion identificada como produccion mantiene APP_DEBUG habilitado, lo que puede exponer detalles internos ante errores.	backend/vercel.json:13-15
ALTO	Tests potencialmente conectados a una base real	La configuracion SQLite de phpunit.xml esta comentada y no se observo un entorno de testing claramente aislado. Ejecutar pruebas podria afectar datos externos si la configuracion activa apunta a una BD real.	backend/phpunit.xml
ALTO	Detalle de producto puede romper React	El detalle busca el producto dentro de una lista limitada a 100. Si el ID no existe o corresponde a un registro fuera de esa lista, autoparte puede quedar undefined y luego se accede a autoparte.foto.	useAutopartes.js:12; DetalleAutoparte.jsx:23-25, 128
MEDIO	Rutas React administrativas sin guards	Se puede navegar directamente a /autoparts/crear y /autoparts/id/editar. En combinacion con la falta de autorizacion del backend, un Cliente podria alcanzar el ABM.	App.jsx:26, 30
MEDIO	Controladores API reutilizados desde Blade	Rutas web reutilizan controladores que devuelven JSON. Por ejemplo, /autoparts puede terminar devolviendo un paginator JSON en lugar de renderizar la vista Blade esperada.	web.php:21-23; AutopartController.php:14-40
MEDIO	Login y registro Blade devuelven JSON	El flujo web administrativo no redirige adecuadamente porque los controladores de autentificacion estan planteados para respuestas JSON.	AuthController.php:50-58, 75-82
MEDIO	Dashboards Blade faltantes	HomeController referencia empleado.dashboard y cliente.dashboard, pero esas vistas no estan presentes.	HomeController.php:14-21
MEDIO	AdminSeeder no asigna rol Spatie	El usuario de AdminSeeder recibe role=Empleado en columna, pero no necesariamente el rol administrado por Spatie. El login API consulta el rol desde Spatie y puede devolver null.	AdminSeeder.php:12-17; AuthController.php:79
MEDIO	Archivos de imagen huérfanos	Al borrar una autoparte no se eliminan necesariamente sus imagenes del storage, generando archivos no referenciados.	AutopartController.php:137-145
MEDIO	Imagenes acopladas a Laravel /storage	React construye directamente URLs de /storage. Esto hace que el frontend conozca detalles del backend y dificulta mover almacenamiento o backend sin cambios en el cliente.	Autopartes.jsx:171; DetalleAutoparte.jsx:136

4. Detalle de los bloqueantes principales

4.1 Autorizacion y roles

La combinacion de registro publico como Empleado, ausencia de autorizacion por rol en la API y rutas administrativas accesibles desde el cliente convierte el control de permisos en el mayor riesgo del proyecto. La seguridad debe imponerse siempre en el servidor. La interfaz puede ocultar botones, pero eso no impide que una solicitud HTTP sea enviada manualmente.

- El registro publico deberia crear unicamente usuarios Cliente.
- POST, PUT/PATCH y DELETE de autopartes deberian exigir explicitamente el rol Empleado o un permiso equivalente.
- Las rutas Blade deben utilizar exactamente el mismo identificador de rol que se almacena / asigna.
- El grupo Cliente no debe contener operaciones administrativas sobre productos.

4.2 Backend Blade

La consigna no pide solamente que existan archivos Blade: el backend debe ser un sistema utilizable por administradores o personal de la empresa. En el estado revisado, ese flujo no esta completo porque existen rutas a metodos inexistentes, controladores web que retornan JSON, autenticacion sin redireccion web y dashboards referenciados pero faltantes.

Acciones de rutas detectadas como inexistentes:

- CarritoController@edit
- PedidoController@store
- CompraController@pagar
- CompraController@comprar
- AutopartController@create
- AutopartController@edit (referenciada en dos rutas)

IMPACTO SOBRE LA CONSIGNA

Mientras estas rutas y flujos no esten corregidos, no puede considerarse completo el checkpoint de Backend con Blade, aunque la API y el frontend React tengan funcionalidad.

4.3 Transaccion de compra y stock

El flujo de compra crea el Pedido y despues valida stock. Ante falta de stock se devuelve un response con error desde dentro de DB::transaction. Retornar una respuesta no equivale a lanzar una excepcion; por lo tanto, la transaccion puede considerarse exitosa y persistir parte del pedido aunque la operacion global haya fallado.

- Validar disponibilidad de todos los items antes de crear el pedido.
- Ejecutar la creacion de pedido, detalles y descuento de stock dentro de una unica transaccion atomica.
- Ante cualquier inconsistencia, lanzar una excepcion o provocar rollback de forma explicita.
- Considerar bloqueo de filas / estrategia de concurrencia para evitar vender el mismo stock simultaneamente.

4.4 Produccion y configuracion

La presencia de localhost en la URL de API, CORS e imagenes hace que la configuracion analizada no sea portable a un entorno real. Ademas, APP_DEBUG activo y archivos de entorno dentro del paquete aumentan el riesgo operativo y de exposicion de informacion.

5. Verificaciones automaticas realizadas

Prueba	Resultado	Detalle
Sintaxis PHP	OK	63 archivos PHP revisados sin errores sintacticos.
Carga de Laravel / listado de rutas	OK	Laravel llego a listar 41 rutas.
Chequeo ruta -> metodo	FALLA	Se encontraron 7 entradas de ruta que apuntan a metodos inexistentes.
Sintaxis JS / JSX	OK	18 archivos JS/JSX parseados sin errores sintacticos.
PHPUnit	NO COMPLETADO	El entorno de revision no contaba con extensiones PHP dom, mbstring, xml y xmlwriter requeridas.
npm run build / npm run lint	NO COMPLETADO	La instalacion de dependencias agoto el tiempo disponible del entorno; no se cuenta como fallo del proyecto.

LIMITACION DE COBERTURA

Los tests existentes no alcanzan para demostrar ausencia de bugs. Se observo una cobertura minima: un test `assertTrue(true)` y otro que solo comprueba que la ruta / responda 200. Ademas, se evito ejecutar pruebas contra la base PostgreSQL configurada para no arriesgar modificaciones sobre una base remota.

6. Otros errores generales y deuda tecnica

Area / archivo	Observacion
RoleKeyMiddleware.php	El archivo no comienza con <code><?php</code> , por lo que no es un archivo PHP valido en su forma actual.
Middleware de Spatie	En Kernel.php se observan namespaces antiguos o incompatibles con versiones actuales del paquete; conviene validar la version instalada y los alias registrados.
Dependencias frontend	Existe una dependencia escrita como bootstrap ademas de bootstrap; el paquete mal escrito aparece como deprecated.
Ruta /home	Contiene codigo de plantilla y un enlace a /about que no existe, señal de contenido residual del scaffolding.
Borrado de productos	Conviene eliminar tambien los archivos fisicos de imagen asociados para evitar acumulacion de storage.

7. Diferencias respecto del alcance declarado en README

Estos puntos no necesariamente son incumplimientos directos del PDF de la materia, pero si representan diferencias entre el alcance declarado por el propio proyecto y la implementacion revisada:

- Productos aleatorios o recomendados: no se encontro una implementacion equivalente completa.
- Categorías y búsqueda / filtrado: no se encontro el alcance prometido de forma completa.
- Dirección de envío: no se observo un flujo completo acorde con lo declarado.
- Integración real de pagos: los medios de pago del frontend se almacenan como un valor o string; no se observo integración con una pasarela de pagos.

8. Plan de correccion recomendado antes de entregar

- 1. Cerrar la autorizacion.** Registro publico solo como Cliente; ABM API exclusivo de Empleado; sacar update/delete del grupo Cliente; aplicar middleware/polices en backend.
- 2. Completar el backend Blade.** Separar controladores web de controladores API, implementar los metodos faltantes, renderizar vistas y corregir login/registro/dashboard administrativo.
- 3. Corregir la transaccion de compra.** Validar stock antes de persistir, garantizar atomicidad y rollback, y contemplar concurrencia.
- 4. Preparar produccion.** Eliminar localhost del codigo, usar variables de entorno para API/assets, configurar CORS real y desactivar APP_DEBUG.
- 5. Implementar logout real.** Revocar el token en backend y eliminar auth_token y datos de sesion del navegador.
- 6. Corregir detalle de producto.** Consultar /autoparts/{id} directamente y manejar estados loading/not-found/error sin acceder a propiedades de undefined.
- 7. Agregar tests funcionales.** Cubrir permisos/roles, CRUD, carrito, stock, compra, pedidos, autenticacion y escenarios de error con una base de testing aislada.

9. Checklist minimo para considerar la entrega lista

- ☐ Un usuario Cliente no puede crear, editar ni eliminar autopartes aunque invoque la API manualmente.
- ☐ El registro publico nunca puede otorgar el rol Empleado.
- ☐ El backend Blade permite a un Empleado iniciar sesion y hacer ABM completo sin depender del frontend React.
- ☐ No existen rutas a metodos de controlador inexistentes.
- ☐ Una compra con stock insuficiente no crea Pedido ni DetallePedido parcial.
- ☐ Una compra valida descuenta stock exactamente una vez y de forma atomica.
- ☐ Logout revoca o invalida el token y el navegador deja de enviarlo.
- ☐ Frontend y backend funcionan con URLs de produccion sin referencias a 127.0.0.1.
- ☐ APP_DEBUG esta desactivado y los secretos no forman parte del repositorio / paquete de entrega.
- ☐ Los tests se ejecutan sobre una base aislada y cubren los flujos criticos.
- ☐ El detalle de producto maneja IDs inexistentes y productos fuera de la primera pagina de resultados.
- ☐ La aplicacion se prueba visualmente en desktop y mobile con el build de produccion.

10. Conclusion

El proyecto tiene una base funcional considerable: productos, fotos, carrito, pedidos, React, Laravel y una API REST estan presentes. No obstante, la autorizacion y el backend Blade son bloqueantes directos frente a la consigna, y la transaccion de compra representa un riesgo de consistencia de datos. La configuracion de produccion y la cobertura de tests tambien deben fortalecerse.

ESTADO RECOMENDADO

No presentar todavia como entrega final. Corregir primero los hallazgos criticos, validar el flujo Blade de punta a punta y ejecutar una bateria de pruebas funcionales en un entorno aislado. Luego realizar un nuevo pase de revision.

Nota: una revision estatica y las pruebas ejecutadas pueden detectar muchos problemas, pero ningun analisis razonable permite garantizar "ausencia absoluta de bugs". El objetivo debe ser reducir el riesgo mediante pruebas, controles de permisos, manejo correcto de errores y validacion de los flujos principales.